

1. Visão geral da API Flask

Entenda, de forma simples, o que esse código faz e como a API funciona.



Explicação simples:

Este código cria uma pequena API com Flask.
Uma API é como um atendente digital: recebe pedidos e devolve respostas.

Trecho do código

```
1 from flask import Flask, jsonify, request
2 app = Flask(__name__)
3 if __name__ == "__main__":
4     app.run(debug=True, port=5000)
```



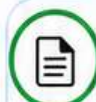
Flask

É a ferramenta principal.
Cria a sua aplicação (API).



jsonify

Transforma dados Python em JSON (formato usado na internet) para enviar como resposta.



request

Permite acessar os dados que o usuário envia na requisição (pedido).



app.run

Inicia a API.
Deixa o servidor rodando para receber pedidos na porta 5000.

API Flask

Usuário ou Aplicação



Envia um pedido (requisição) para a API

PEDIDO
(REQUISIÇÃO)

Ex.: GET, POST,
PUT, DELETE

RESPOSTA
(DADOS EM JSON)



Recebe o pedido,
processa e devolve
uma resposta

Produtos (exemplos)



Notebook

R\$ 3.499,00

ID: 1



Mouse

R\$ 89,90

ID: 2



Teclado

R\$ 149,90

ID: 3

O que essa API permite fazer?

1



Listar

Retorna todos os produtos.

GET /produtos

2



Buscar

Busca um produto pelo ID.

GET /produtos/<id>

3



Criar

Cadastra um novo produto.

POST /produtos

4



Atualizar

Atualiza os dados de um produto.

PUT /produtos/<id>

5



Remover

Remove um produto pelo ID.

DELETE /produtos/<id>



Em resumo

Você faz um pedido → a API Flask processa → busca ou altera os dados → e envia de volta a resposta.

Simples, organizada e poderosa!

2.

Banco de dados em memória – lista de produtos



Em Python, podemos guardar dados em uma lista.

Aqui, usamos uma lista para simular um pequeno banco de dados.



Explicação simples:

- A lista **produtos** funciona como um “banco de dados” temporário.
- Ela fica guardada apenas na memória do servidor enquanto o programa está rodando.
- Cada produto é um dicionário (informações em pares: chave e valor).
- É fácil de entender e ótima para aprender!

</> Trecho do código

```
1 produtos = [  
2     {"id": 1, "nome": "Notebook", "preco": 3500.00, "quantidade": 10},  
3     {"id": 2, "nome": "Mouse", "preco": 80.00, "quantidade": 50},  
4     {"id": 3, "nome": "Teclado", "preco": 150.00, "quantidade": 30},  
5 ]  
6  
7 proximo_id = 4
```

Produtos cadastrados na lista produtos

1 Notebook



id: 1
nome: Notebook
preco: 3500.00
quantidade: 10

2 Mouse



id: 2
nome: Mouse
preco: 80.00
quantidade: 50

3 Teclado



id: 3
nome: Teclado
preco: 150.00
quantidade: 30



O que é proximo_id?

proximo_id guarda o próximo número de ID que será usado quando um novo produto for cadastrado.

No exemplo, o próximo ID disponível é 4.

Quando criamos um novo produto, usamos o valor de **proximo_id** e depois aumentamos para o próximo.

1 2 3 4 próximo ID disponível



Atenção!

Como os dados estão em memória, eles podem desaparecer se o servidor for reiniciado ou se o programa for encerrado.



Se o servidor reiniciar, a lista **produtos** volta vazia.



Para manter os dados para sempre, precisamos salvá-los em um banco de dados real (como SQLite, PostgreSQL, MySQL etc.).



Em resumo



A lista **produtos** é um banco de dados simples e temporário.



Cada produto tem **id**, **nome**, **preco** e **quantidade**.



Os dados ficam apenas na memória e podem sumir ao reiniciar.

3. Função buscar_produto(id)

Busca um produto pelo ID dentro da lista de produtos.



Explicação simples:

Essa função recebe um ID e procura, dentro da lista de produtos, aquele que tem o ID igual.

Se encontrar, ela devolve o produto.

Se não encontrar, devolve **None** (que significa “não encontrou nada”).

Lista de produtos



ID: 1

Notebook
R\$ 3.499,00



ID: 2

Mouse
R\$ 89,90



ID: 3

Teclado
R\$ 199,90



ID: 4

Fone de ouvido
R\$ 199,90



Trecho do código

```
def buscar_produto(id):  
    for produto in produtos:  
        if produto["id"] == id:  
            return produto  
    return None
```



Passo a passo

1



Recebe o ID

A função recebe um ID como entrada.

2



Procura na lista

Percorre a lista de produtos procurando o produto cujo ID é igual ao informado.

3



Retorna o produto ou None

Se encontrar, devolve o produto.
Se não encontrar, devolve None (não encontrou nada).



Exemplo

Chamada da função:

buscar_produto(2)



Resultado:



ID: 2

Mouse
R\$ 89,90

A função encontrou o produto com ID 2 e o retornou.



E se não encontrar?

Se nenhum produto tiver o ID informado, a função retorna:

None



Isso significa:

“não encontrou nada”



None não é um erro.
É apenas a forma da função dizer que não achou nenhum produto.



Em resumo

A função buscar_produto(id) recebe um ID, procura na lista de produtos e:



devolve o produto se encontrar



devolve **None** se não encontrar.



Essa função é muito útil para encontrar informações específicas rapidamente!



4. GET/produtos — listar todos os produtos

Esta rota retorna a lista completa de todos os produtos cadastrados.



Explicação simples:

GET é um tipo de pedido que serve para **LER** informações.

Nesta rota, ele pede a lista de todos os produtos e a API devolve essa lista.

</> Trecho do código

```
@app.route("/produtos", methods=["GET"])
def listar_produtos():
    return jsonify(produtos), 200
```

1. Usuário



O usuário faz um pedido para ver todos os produtos.

2. Pedido (requisição)

GET /produtos

O método GET pede para LER os dados dos produtos.

API Flask



3. Resposta

A API devolve a lista completa de todos os produtos em formato JSON.

Lista de produtos (Resposta)



Notebook
R\$ 3.499,00



Mouse
R\$ 89,90



Teclado
R\$ 149,90



O que significa jsonify(produtos)?

jsonify(produtos) transforma a lista de produtos (Python) em JSON, que é um formato de texto facilmente entendido por qualquer sistema.

200

O que significa o código 200?

O código 200 (OK) indica que o pedido foi bem-sucedido e que os dados foram retornados corretamente.



Exemplo de resposta (JSON)

```
[
  { "id": 1, "nome": "Notebook", "preco": 3499.00 },
  { "id": 2, "nome": "Mouse", "preco": 89.90 },
  { "id": 3, "nome": "Teclado", "preco": 149.90 }
]
```

Obs.: A ordem e os campos podem variar conforme seu projeto.



Quando usar?

- ✓ Quando o usuário quer ver todos os produtos cadastrados.
- ✓ Para preencher listas, tabelas ou catálogos na aplicação.
- ✓ Sempre que for necessário **LER** os dados de produtos.



Em resumo

Você faz um pedido **GET** para **/produtos** → a API procura a lista de produtos → converte para **JSON** → retorna com sucesso (**200**) → você recebe a lista completa! 🚀

5

GET /produtos/<id> – buscar produto específico

Esta rota retorna apenas um produto, com base no ID informado na URL.



Explicação simples:

Você envia uma requisição GET informando o ID do produto na URL. A API procura esse produto no banco de dados e retorna somente ele. Se o produto não existir, a API informa que não encontrou.

</> Trecho do código

```
1 @app.route("/produtos/<int:id>", methods=["GET"])
2 def obter_produto(id):
3     produto = buscar_produto(id)
4     if produto is None:
5         return jsonify({"erro": "Produto não encontrado"}), 404
6     return jsonify(produto), 200
```

1 Produto encontrado

GET /produtos/2



Resposta da API

200 OK

```
{
  "id": 2,
  "nome": "Mouse Sem Fio",
  "preco": 89.90,
  "estoque": 15
}
```

A API encontrou o produto com ID 2 e retornou todos os seus dados.

2 Produto não encontrado

GET /produtos/999



Resposta da API

404 Not Found

```
{
  "erro": "Produto não encontrado"
}
```

A API não encontrou nenhum produto com ID 999 e retornou um erro 404.



Entenda os códigos de resposta

200 OK

Tudo certo! A requisição foi bem-sucedida e a API retornou o produto solicitado.



404 Not Found

O produto com o ID informado não existe na base de dados.



Exemplo de rota

GET

/produtos/2

Busca o produto com ID igual a 2.

GET

/produtos/999

Busca o produto com ID igual a 999.



Exemplo de resposta (sucesso - 200 OK)

```
{
  "id": 2,
  "nome": "Mouse Sem Fio",
  "preco": 89.90,
  "estoque": 15
}
```

A API retorna os dados do produto em formato JSON. Cada produto é único e identificado pelo seu ID.



Dica de uso

Use sempre o ID correto na URL. IDs diferentes retornam produtos diferentes!



Em resumo

Você envia um GET para /produtos/<id> → a API procura o produto pelo ID → se encontrar, retorna 200 OK com os dados → se não encontrar, retorna 404 Not Found.

6.

POST /produtos — criar novo produto

Usamos POST para enviar dados e criar um novo produto na API.



Explicação simples:

O método **POST** envia informações para o servidor. Quando enviamos os dados de um produto para a rota **/produtos**, a API cria esse novo produto, gera um **ID** automático e o adiciona à lista.

</> Trecho do código

```
@app.route("/produtos", methods=["POST"])
def criar_produto():
    global proximo_id
    dados = request.get_json()

    novo_produto = {
        "id": proximo_id,
        "nome": dados["nome"],
        "preco": dados["preco"],
        "quantidade": dados["quantidade"],
    }

    produtos.append(novo_produto)
    proximo_id += 1

    return jsonify(novo_produto), 201
```

1 Enviando dados (requisição)

O cliente envia um **POST** para **/produtos** com os dados do novo produto (em JSON).



POST
/produtos
→
JSON

```
{
  "nome": "Teclado Mecânico",
  "preco": 249.90,
  "quantidade": 15
}
```



request.get_json()

Lê o corpo da requisição (JSON) e transforma em um dicionário Python (dados).



append

Adiciona o novo produto ao final da lista **produtos**.

201

201 (Created)

Código de resposta que indica: o recurso foi criado com sucesso.

2 Resposta da API (novo produto criado)

A API cria o produto, gera um ID automático e retorna o produto criado com status **201**.



```
{
  "id": 7,
  "nome": "Teclado Mecânico",
  "preco": 249.90,
  "quantidade": 15
}
```

201 Created



Antes e depois na lista de produtos

ANTES

Lista de produtos

- 1 Notebook Gamer R\$ 3.499,00
- 2 Mouse Sem Fio R\$ 89,90
- 3 Fone Bluetooth R\$ 129,90

DEPOIS

Lista de produtos

- 1 Notebook Gamer R\$ 3.499,00
- 2 Mouse Sem Fio R\$ 89,90
- 3 Fone Bluetooth R\$ 129,90
- 7 Teclado Mecânico R\$ 249,90

Novo!

Em resumo



Cliente envia **POST /produtos** com os dados.

1



A API lê os dados, cria o produto e adiciona à lista.

3



A API retorna o produto criado com status **201**.

4



O novo produto já faz parte da lista!



Lembre-se:

POST envia dados e cria algo novo no servidor.

7.

Validação dos campos obrigatórios no POST

Antes de criar um produto, o código verifica se os dados obrigatórios foram enviados.



Explicação simples:

Quando você envia um POST para criar um produto, o sistema confere se todos os campos obrigatórios estão presentes no corpo da requisição. Se algum campo estiver faltando, ele **não cria o produto** e retorna um **erro 400** explicando o que está faltando.

Trecho do código



```
if not dados:
    return jsonify({"erro": "Corpo da requisição inválido"}), 400

campos_obrigatorios = ["nome", "preco", "quantidade"]
for campo in campos_obrigatorios:
    if campo not in dados:
        return jsonify({"erro": f"Campo obrigatório ausente: {campo}"}), 400
```



Exemplo completo (ok)

Todos os campos obrigatórios foram enviados.

Requisição (corpo JSON)

```
{
  "nome": "Caderno",
  "preco": 19.90,
  "quantidade": 10
}
```



Produto criado com sucesso!



Exemplo incompleto (erro)

Faltou o campo obrigatório **"preco"**.

Requisição (corpo JSON)

```
{
  "nome": "Caderno",
  "quantidade": 10
}
```



Não foi criado.
Faltou um campo obrigatório.



O que é campo obrigatório?

É um dado que o sistema precisa receber para poder realizar a ação. No nosso exemplo, os campos obrigatórios são:

nome

preco

quantidade

400

O que significa o erro 400?

É o código de status HTTP que significa: **"Bad Request"** (Requisição inválida). Ou seja, o pedido foi feito errado, por exemplo, porque faltou um dado necessário.

Exemplo de requisição inválida (corpo JSON)

```
{
  "nome": "Lápis",
  "quantidade": 5
}
```



Resposta de erro (HTTP 400) (JSON)

```
{
  "erro": "Campo obrigatório ausente: preco"
}
```



Em resumo (para leigos):

1



Você envia os dados para criar o produto.



2



O sistema verifica se os **campos obrigatórios** estão presentes.



3



Se estiver tudo certo, o produto é criado.



4



Se faltar algo, o sistema retorna erro 400 e diz qual campo está faltando.



Dica

Sempre envie todos os campos obrigatórios para evitar erros!



Dados corretos geram resultados corretos! 🚀

8.

PUT /produtos/<id> — atualizar produto existente

Altera os dados de um produto que já existe no sistema.



Explicação simples:

O método PUT é usado para atualizar um produto que já existe. Você envia os dados que deseja mudar e o sistema substitui **apenas esses campos**. Os campos que não forem enviados permanecem iguais.



Trecho do código

```
@app.route("/produtos/<int:id>", methods=["PUT"])
def atualizar_produto(id):
    produto = buscar_produto(id)

    if produto is None:
        return jsonify({"erro": "Produto não encontrado"}), 404

    dados = request.get_json()
    if not dados:
        return jsonify({"erro": "Corpo da requisição inválido"}), 400

    if "nome" in dados:
        produto["nome"] = dados["nome"]
    if "preco" in dados:
        produto["preco"] = dados["preco"]
    if "quantidade" in dados:
        produto["quantidade"] = dados["quantidade"]

    return jsonify(produto), 200
```

Exemplo: antes da atualização



Mouse

ID: 42

Preço: R\$ 89,90

Quantidade: 10

Categoria: Periféricos



Exemplo: depois da atualização



Mouse

ID: 42

Preço: R\$ 74,90

Quantidade: 25

Categoria: Periféricos



Apenas os campos enviados foram alterados (**preço e quantidade**). Os demais campos permanecem iguais.



Exemplo de requisição (atualizando preço e quantidade)

Método: **PUT**

URL: **/produtos/42**

Cabeçalho:

Content-Type: application/json

Corpo da requisição (JSON):

```
{
  "preco": 74.90,
  "quantidade": 25
}
```



E se o produto não existir?

Se o ID informado não existir no sistema, a API retorna:

404 - Produto não encontrado

Exemplo de resposta:

```
{
  "erro": "Produto não encontrado"
}
```



Em resumo



Você envia um PUT para **/produtos/<id>**



Envia apenas os dados que deseja alterar



A API atualiza somente esses campos



Você recebe o produto atualizado **com sucesso!**



Dica importante: Para usar o PUT, o produto precisa existir. Caso contrário, você receberá um erro 404.

9.

DELETE /produtos/<id> — remover produto



Use DELETE para remover um produto da lista pelo seu ID.



Explicação simples:

O método **DELETE** remove um produto da lista usando o ID informado na URL. Se o produto existir, ele é removido e a API retorna uma mensagem de sucesso. Se não existir, a API retorna um erro 404 (não encontrado).



Trecho do código

```
@app.route("/produtos/<int:id>", methods=["DELETE"])
def deletar_produto(id):
    produto = buscar_produto(id)
    if produto is None:
        return jsonify({"erro": "Produto não encontrado"}), 404

    produtos.remove(produto)
    return jsonify({"mensagem": f"Produto {id} removido com sucesso"}), 200
```

1. Requisição

Método: **DELETE**

URL: /produtos/3



A API recebe o pedido para remover o produto com ID 3.

2. Antes da remoção

Lista de produtos

- 1  **Notebook**
R\$ 3.499,00
- 2  **Mouse**
R\$ 89,90
- 3  **Teclado**
R\$ 149,90
- 4  **Fone**
R\$ 199,90



3. Depois da remoção

Lista de produtos atualizada

- 1  **Notebook**
R\$ 3.499,00
- 2  **Mouse**
R\$ 89,90
- 4  **Fone**
R\$ 199,90

✓ O produto com ID 3 foi removido da lista.



O que faz o remove?

O comando `produtos.remove(produto)` remove o item encontrado da lista `produtos`.

É como tirar um elemento de uma lista.

200

O que significa 200?

Código de status 200 (OK) indica que a operação foi realizada com sucesso. Ou seja, o produto foi removido.

404

O que significa 404?

Código de status 404 (Not Found) indica que o produto com o ID informado não foi encontrado. Nada é removido.



Exemplo de resposta (sucesso)

```
{
  "mensagem": "Produto 3 removido com sucesso"
}
```

Status HTTP: 200

Tipo: application/json



Em resumo

Você envia
DELETE /produtos/<id>



A API procura
o produto pelo ID



Se existir,
remove da lista



Retorna 200
sucesso



Se não existir,
retorna 404

DELETE é usado para remover algo com segurança informando o ID do item. 🚀

10. Dicionários em Python

Aprenda, de forma simples, como guardar informações usando pares de chave e valor.



Explicação simples:

Um dicionário em Python é uma estrutura usada para guardar dados em pares de **chave** e valor.

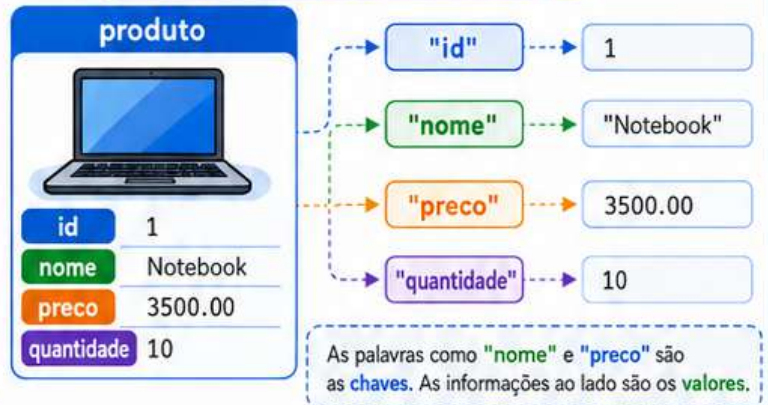
Pense como uma ficha de identificação ou um formulário de produto: cada informação (ex.: nome, preço) é a **chave**, e o dado correspondente (ex.: notebook, 3500.00) é o **valor**.

As chaves nos ajudam a encontrar as informações rapidamente!

Trecho do código

```
1 produto = {  
2     "id": 1,  
3     "nome": "Notebook",  
4     "preco": 3500.00,  
5     "quantidade": 10  
6 }
```

Como ler esse dicionário?



Como acessar um valor?

Usamos a chave entre colchetes para obter o valor guardado.

```
>_ produto["nome"] → Resultado: "Notebook"
```

```
>_ produto["preco"] → Resultado: 3500.00
```



Adicionar ou alterar informações

Podemos adicionar novas chaves ou alterar valores existentes.

```
1 produto["marca"] = "Dell"  
2 produto["quantidade"] = 8
```

- A primeira linha **adiciona** uma nova informação (marca: Dell).
- A segunda linha **altera** o valor de uma chave já existente (quantidade passa a ser 8).

O que um dicionário permite fazer?

1 Guardar dados organizados



Agrupa informações relacionadas em um só lugar.

2 Acessar valores pela chave



Encontre rapidamente a informação que você precisa.

3 Atualizar informações



Altere valores facilmente quando for necessário.

4 Adicionar novos campos



Inclua novas chaves e expanda seus dados sem complicação.

5 Representar objetos do mundo real



Perfeito para produtos, alunos, usuários e muito mais!



Exemplo do dia a dia

Um dicionário pode representar uma ficha de aluno ou um formulário de produto. Veja um exemplo de ficha de aluno:

```
1 aluno = {  
2     "nome": "Caio",  
3     "idade": 7,  
4     "turma": "1º ano"  
5 }
```



nome	Caio
idade	7
turma	1º ano

Esse dicionário guarda várias informações sobre o mesmo aluno de forma organizada e fácil de usar.



Em resumo

Dicionários em Python guardam informações em pares de chave e valor.

Eles ajudam a organizar dados, acessar informações rapidamente

e representar objetos como produtos, alunos e usuários. **Simples, prático e poderoso!**